

Build Justification (for Demeter Card which can build twice)

Determining a Valid Build

Where it's checked:

The current `TurnAction` (usually a `DefaultBuildAction`) determines whether a build is valid, using the `getValidTiles(Game)` method.

How it's checked:

- When the player clicks a `Tile`, `Game.handleTilePress(Tile)` calls the current `TurnAction`'s `attempt(Tile, Game)` method.
- Inside `attempt()`, the `TurnAction` calls its own `getValidTiles(Game)` method to get the set of allowed tiles to click on.
- `getValidTiles(Game)` (in `DefaultBuildAction`) checks if the following conditions are true:
 - The `Tile` is adjacent to the `Worker`'s current `Tile`,
 - The `Tile` is unoccupied,
 - The `Tile` does not have a dome.

Design Justification:

- **Single Responsibility Principle:** `Game` is responsible for high-level game flow (turns, switching phases, etc.), while `TurnAction` subclasses are responsible for validating and performing specific player actions.
- **Information Expert Principle:** `TurnAction` knows what kind of action is being taken and what constraints apply, making it the correct place to validate builds.
- **Open-Closed Principle:** New god cards (like Demeter, Hephaestus) can override or modify `getValidTiles()` without changing `Game`, allowing easy extensions.

Alternatives Considered:

1. **Alternative:** Move build validation into `Game`.
 - **Trade-off:** Would require `Game` to understand the specific rules of different actions and god cards, which is not very flexible as different cards have different sets of legal moves.
2. **Alternative:** Have a separate class within each `TurnAction` to determine move legality.
 - **Trade-off:** Unnecessary added complexity.

Performing the Build

Where it's executed:

Building is performed by the current `TurnAction`, which is part of the player's `GodCard`, when `Game.handleTilePress(Tile)` calls it.

How it's executed:

- `Game.handleTilePress(Tile)` calls the current `TurnAction`'s `attempt(Tile, Game)`.
- Inside `attempt()`, if the tile is valid, the `TurnAction` executes its action.
- For Demeter's first build:
 - `DemeterFirstBuild` calls `tile.buildTower()` with a delegate `DefaultBuildAction`.
- For Demeter's second build:
 - `DemeterSecondBuild` similarly calls `tile.buildTower()` with a delegate `DefaultBuildAction`, but after checking that it is not the same `Tile` as the first.

Design Justification:

- **Encapsulation:** `Game` controls high-level flow, but delegates low-level state changes (like modifying `Tower`) to `Tile`.
- **Delegation:** The First and Second build `TurnActions` delegate the actual tower building to the default behavior, because this is the most robust if something about the default behavior changes.

Alternatives Considered:

1. **Having exactly one move phase followed by exactly one build phase:** This was my original design, but it proved to be ineffective for cards such as `DemeterCard` which had multiple build phases. I could have done some weird code to make it work, but I found it easier to create new classes of `TurnAction`.